

BASE DE DATOS II

Práctica Guiada de Laboratorio

Introducción a Apache Cassandra DB

Tema	Bases de datos NoSQL, Cassandra, Keyspace, tablas orientadas a consultas, CQL
Motor de base de datos	Apache Cassandra
Herramienta sugerida	CMD/PowerShell, cqlsh y Apache Cassandra instalado localmente
Modalidad	Individual
Duración estimada	90 a 120 minutos
Producto esperado	Script CQL, capturas de ejecución y breve análisis

Nota para el docente: La práctica está diseñada para estudiantes universitarios que recién inician Cassandra. Se recomienda explicar primero que Cassandra no se modela igual que PostgreSQL o SQL Server: en Cassandra se diseñan las tablas según las consultas que se necesitan responder.

1. Logro de aprendizaje

Al finalizar la práctica, el estudiante será capaz de iniciar Cassandra, ingresar a cqlsh, crear un keyspace, crear tablas orientadas a consultas, insertar datos, consultar información con SELECT, actualizar y eliminar registros, y comprender el uso de claves primarias en Cassandra.

2. Caso práctico: Biblioteca universitaria

La biblioteca de una universidad necesita registrar usuarios, libros y préstamos. Como el sistema debe consultar rápidamente los préstamos de cada estudiante y el estado de los préstamos, se usará Cassandra DB. A diferencia de una base relacional, se crearán tablas pensadas directamente en las preguntas que el sistema necesita responder.

Consultas que resolverá el sistema: 1) Ver los préstamos de un usuario. 2) Ver los préstamos según su estado. 3) Buscar libros por autor. 4) Ver el detalle de un préstamo específico.

3. Requisitos previos

- Apache Cassandra instalado en la computadora.
- Java compatible configurado correctamente en JAVA_HOME. Para Cassandra 3.x normalmente se recomienda Java 8.

- Variable CASSANDRA_HOME configurada si corresponde.
- CMD o PowerShell disponible.
- Carpeta de Cassandra ubicada, por ejemplo, en C:\cassandra.

Importante: Si al ejecutar Cassandra aparece el error JAVA_HOME environment variable must be set, significa que Java no está configurado. Si aparece Unrecognized VM option UseParNewGC, normalmente Cassandra está intentando usar una versión de Java demasiado nueva.

4. Iniciar Cassandra

Abre una ventana CMD como administrador. Luego ingresa a la carpeta bin de Cassandra.

```
cd c:\cassandra\bin
```

Ejecuta Cassandra:

```
cassandra
```

Si inicia correctamente, deja esta ventana abierta. Cassandra funciona como servidor, por eso debe permanecer ejecutándose mientras trabajas.

Resultado esperado: Debe mostrarse información de inicio del servidor Cassandra. Si aparecen muchos mensajes, no necesariamente es error. Lo importante es que la consola no cierre el proceso inmediatamente ni muestre Fatal exception.

5. Abrir cqlsh

Abre una segunda ventana CMD. No cierres la primera. En la nueva ventana entra nuevamente a la carpeta bin y ejecuta cqlsh.

```
cd c:\cassandra\bin  
cqlsh
```

Si todo está correcto, aparecerá una consola similar a:

```
Connected to Test Cluster at 127.0.0.1:9042.  
cqlsh>
```

6. Crear el keyspace

En Cassandra, un keyspace es parecido a una base de datos en PostgreSQL o SQL Server. Dentro del keyspace se crean las tablas.

```
CREATE KEYSPACE biblioteca_universitaria  
WITH replication = {  
  'class': 'SimpleStrategy',  
  'replication_factor': 1  
};
```

Luego selecciona el keyspace:

```
USE biblioteca_universitaria;
```

Explicación: SimpleStrategy se usa en prácticas locales o entornos de aprendizaje. replication_factor = 1 significa que se guardará una copia de los datos. En ambientes reales se usan más réplicas y estrategias de replicación según el centro de datos.

7. Verificar keyspaces existentes

```
DESCRIBE KEYSPACES;
```

Debe aparecer el keyspace biblioteca_universitaria junto con otros keyspaces internos de Cassandra.

8. Diseño de tablas orientadas a consultas

En Cassandra no se recomienda diseñar primero muchas tablas normalizadas para luego unirlos. Cassandra no trabaja como una base relacional con JOIN tradicionales. Se crean tablas según las consultas principales del sistema.

Consulta necesaria	Tabla propuesta	Clave principal recomendada
Ver préstamos de un usuario	prestamos_por_usuario	id_usuario, fecha_prestamo
Ver préstamos por estado	prestamos_por_estado	estado, fecha_prestamo
Ver libros por autor	libros_por_autor	autor, titulo_libro
Ver detalle de un préstamo	prestamo_detalle	id_prestamo

9. Crear tabla prestamos_por_usuario

Esta tabla permitirá responder rápidamente la pregunta: ¿qué préstamos tiene un usuario?

```
CREATE TABLE prestamos_por_usuario (  
  id_usuario text,  
  fecha_prestamo date,  
  id_prestamo text,  
  nombre_usuario text,  
  codigo_usuario text,  
  titulo_libro text,  
  autor text,  
  estado text,  
  PRIMARY KEY (id_usuario, fecha_prestamo, id_prestamo)  
) WITH CLUSTERING ORDER BY (fecha_prestamo DESC);
```

Explicación: id_usuario es la clave de partición. Cassandra agrupa los préstamos por usuario. fecha_prestamo e id_prestamo ayudan a ordenar e identificar los préstamos dentro de cada usuario.

10. Crear tabla prestamos_por_estado

Esta tabla permitirá consultar préstamos según su estado: Activo, Devuelto o Vencido.

```
CREATE TABLE prestamos_por_estado (  
    estado text,  
    fecha_prestamo date,  
    id_prestamo text,  
    id_usuario text,  
    nombre_usuario text,  
    titulo_libro text,  
    PRIMARY KEY (estado, fecha_prestamo, id_prestamo)  
) WITH CLUSTERING ORDER BY (fecha_prestamo DESC);
```

Explicación: estado es la clave de partición. Esto permite buscar rápido todos los préstamos que están Activo, Devuelto o Vencido.

11. Crear tabla libros_por_autor

Esta tabla permitirá responder: ¿qué libros tiene registrado un autor?

```
CREATE TABLE libros_por_autor (  
    autor text,  
    titulo_libro text,  
    id_libro text,  
    categoria text,  
    anio_publicacion int,  
    PRIMARY KEY (autor, titulo_libro)  
);
```

Explicación: autor es la clave de partición y titulo_libro permite ordenar los libros del autor dentro de esa partición.

12. Crear tabla prestamo_detalle

Esta tabla permitirá buscar el detalle completo de un préstamo usando su identificador.

```
CREATE TABLE prestamo_detalle (  
    id_prestamo text PRIMARY KEY,  
    id_usuario text,  
    nombre_usuario text,  
    codigo_usuario text,  
    id_libro text,  
    titulo_libro text,  
    autor text,  
    fecha_prestamo date,  
    fecha_devolucion date,  
    estado text  
);
```

13. Insertar datos en las tablas

En Cassandra es normal insertar datos repetidos en varias tablas, porque cada tabla responde una consulta diferente. Esta duplicación controlada ayuda a obtener consultas rápidas.

```
INSERT INTO prestamos_por_usuario
(id_usuario, fecha_prestamo, id_prestamo, nombre_usuario, codigo_usuario, titulo_libro, autor,
estado)
VALUES ('U001', '2026-06-01', 'P001', 'Ana Torres', '2024001', 'Base de Datos Modernas', 'Carlos
Coronel', 'Activo');
```

```
INSERT INTO prestamos_por_usuario
(id_usuario, fecha_prestamo, id_prestamo, nombre_usuario, codigo_usuario, titulo_libro, autor,
estado)
VALUES ('U001', '2026-06-05', 'P002', 'Ana Torres', '2024001', 'Introducción a NoSQL', 'Ana Díaz',
'Devuelto');
```

```
INSERT INTO prestamos_por_usuario
(id_usuario, fecha_prestamo, id_prestamo, nombre_usuario, codigo_usuario, titulo_libro, autor,
estado)
VALUES ('U002', '2026-06-03', 'P003', 'Luis Mendoza', '2024002', 'Apache Cassandra Básico', 'María
Rojas', 'Activo');
```

```
INSERT INTO prestamos_por_usuario
(id_usuario, fecha_prestamo, id_prestamo, nombre_usuario, codigo_usuario, titulo_libro, autor,
estado)
VALUES ('U003', '2026-06-04', 'P004', 'Carla Salazar', '2024003', 'Modelamiento de Datos', 'Carlos
Coronel', 'Vencido');
```

```
INSERT INTO prestamos_por_estado
(estado, fecha_prestamo, id_prestamo, id_usuario, nombre_usuario, titulo_libro)
VALUES ('Activo', '2026-06-01', 'P001', 'U001', 'Ana Torres', 'Base de Datos Modernas');
```

```
INSERT INTO prestamos_por_estado
(estado, fecha_prestamo, id_prestamo, id_usuario, nombre_usuario, titulo_libro)
VALUES ('Devuelto', '2026-06-05', 'P002', 'U001', 'Ana Torres', 'Introducción a NoSQL');
```

```
INSERT INTO prestamos_por_estado
(estado, fecha_prestamo, id_prestamo, id_usuario, nombre_usuario, titulo_libro)
VALUES ('Activo', '2026-06-03', 'P003', 'U002', 'Luis Mendoza', 'Apache Cassandra Básico');
```

```
INSERT INTO prestamos_por_estado
(estado, fecha_prestamo, id_prestamo, id_usuario, nombre_usuario, titulo_libro)
VALUES ('Vencido', '2026-06-04', 'P004', 'U003', 'Carla Salazar', 'Modelamiento de Datos');
```

```
INSERT INTO libros_por_autor
(autor, titulo_libro, id_libro, categoria, anio_publicacion)
VALUES ('Carlos Coronel', 'Base de Datos Modernas', 'L001', 'Base de datos', 2022);
```

```
INSERT INTO libros_por_autor
(autor, titulo_libro, id_libro, categoria, anio_publicacion)
VALUES ('Carlos Coronel', 'Modelamiento de Datos', 'L004', 'Diseño de datos', 2021);
```

```
INSERT INTO libros_por_autor
(autor, titulo_libro, id_libro, categoria, anio_publicacion)
```

```
VALUES ('Ana Díaz', 'Introducción a NoSQL', 'L002', 'NoSQL', 2023);

INSERT INTO libros_por_autor
(autor, titulo_libro, id_libro, categoria, anio_publicacion)
VALUES ('María Rojas', 'Apache Cassandra Básico', 'L003', 'NoSQL', 2024);

INSERT INTO prestamo_detalle
(id_prestamo, id_usuario, nombre_usuario, codigo_usuario, id_libro, titulo_libro, autor,
fecha_prestamo, fecha_devolucion, estado)
VALUES ('P001', 'U001', 'Ana Torres', '2024001', 'L001', 'Base de Datos Modernas', 'Carlos
Coronel', '2026-06-01', '2026-06-15', 'Activo');

INSERT INTO prestamo_detalle
(id_prestamo, id_usuario, nombre_usuario, codigo_usuario, id_libro, titulo_libro, autor,
fecha_prestamo, fecha_devolucion, estado)
VALUES ('P002', 'U001', 'Ana Torres', '2024001', 'L002', 'Introducción a NoSQL', 'Ana Díaz', '2026-
06-05', '2026-06-12', 'Devuelto');

INSERT INTO prestamo_detalle
(id_prestamo, id_usuario, nombre_usuario, codigo_usuario, id_libro, titulo_libro, autor,
fecha_prestamo, fecha_devolucion, estado)
VALUES ('P003', 'U002', 'Luis Mendoza', '2024002', 'L003', 'Apache Cassandra Básico', 'María
Rojas', '2026-06-03', '2026-06-17', 'Activo');

INSERT INTO prestamo_detalle
(id_prestamo, id_usuario, nombre_usuario, codigo_usuario, id_libro, titulo_libro, autor,
fecha_prestamo, fecha_devolucion, estado)
VALUES ('P004', 'U003', 'Carla Salazar', '2024003', 'L004', 'Modelamiento de Datos', 'Carlos
Coronel', '2026-06-04', '2026-06-18', 'Vencido');
```

14. Consultas básicas con SELECT

Actividad 1. Mostrar todos los préstamos de un usuario

```
SELECT * FROM prestamos_por_usuario WHERE id_usuario = 'U001';
```

Explicación: Esta consulta es válida porque usa la clave de partición `id_usuario`. Cassandra puede encontrar rápidamente los registros de ese usuario.

Actividad 2. Mostrar préstamos activos

```
SELECT * FROM prestamos_por_estado WHERE estado = 'Activo';
```

Explicación: La tabla `prestamos_por_estado` fue creada para consultar por estado. Por eso la consulta usa `WHERE estado = Activo`.

Actividad 3. Mostrar libros de un autor

```
SELECT * FROM libros_por_autor WHERE autor = 'Carlos Coronel';
```

Explicación: La consulta muestra todos los libros registrados para un autor específico. En Cassandra es importante consultar usando la clave de partición.

Actividad 4. Buscar detalle de un préstamo

```
SELECT * FROM prestamo_detalle WHERE id_prestamo = 'P001';
```

Explicación: id_prestamo es la clave primaria de prestamo_detalle. Esta consulta permite obtener el detalle completo de un préstamo.

15. Consultas con filtros permitidos

Actividad 5. Consultar préstamos de un usuario desde una fecha

```
SELECT * FROM prestamos_por_usuario  
WHERE id_usuario = 'U001' AND fecha_prestamo >= '2026-06-01';
```

Explicación: La consulta usa la clave de partición id_usuario y luego filtra por fecha_prestamo, que forma parte de la clave de clustering.

Actividad 6. Consultar préstamos activos desde una fecha

```
SELECT * FROM prestamos_por_estado  
WHERE estado = 'Activo' AND fecha_prestamo >= '2026-06-01';
```

Explicación: Primero se filtra por estado, que es la clave de partición. Luego se filtra por fecha_prestamo.

16. Actualización de datos

Actividad 7. Cambiar estado de un préstamo en la tabla de detalle

```
UPDATE prestamo_detalle  
SET estado = 'Devuelto'  
WHERE id_prestamo = 'P001';
```

Verifica el cambio:

```
SELECT * FROM prestamo_detalle WHERE id_prestamo = 'P001';
```

Importante: Si el dato estado también está repetido en otras tablas, debe actualizarse en cada tabla donde se haya duplicado. Esta es una responsabilidad del diseño en Cassandra.

17. Eliminación de datos

Actividad 8. Eliminar un libro por autor

```
DELETE FROM libros_por_autor  
WHERE autor = 'Ana Díaz' AND titulo_libro = 'Introducción a NoSQL';
```

Verifica la eliminación:

```
SELECT * FROM libros_por_autor WHERE autor = 'Ana Díaz';
```

18. Comandos de exploración

Ver tablas del keyspace

```
DESCRIBE TABLES;
```

Ver estructura de una tabla

```
DESCRIBE TABLE prestamos_por_usuario;
```

Ver keyspace actual

```
DESCRIBE KEYSPACE biblioteca_universitaria;
```

Salir de cqlsh

```
EXIT;
```

19. Errores frecuentes y solución

Error frecuente	Causa probable	Solución
JAVA_HOME environment variable must be set	Java no está configurado	Configurar JAVA_HOME con una versión compatible de Java.
Unrecognized VM option UseParNewGC	Java demasiado nuevo para la versión de Cassandra	Usar Java 8 para Cassandra 3.x o una versión compatible.
Connection refused 127.0.0.1:9042	Cassandra no está iniciado	Abrir una ventana CMD y ejecutar cassandra antes de usar cqlsh.
Cannot execute this query as it might involve data filtering	Consulta no usa clave de partición	Diseñar una tabla para esa consulta o consultar por la clave primaria correcta.
InvalidRequest: Missing mandatory PRIMARY KEY part	Falta una parte obligatoria de la clave	Incluir la clave de partición completa en el WHERE.

20. Ejercicios propuestos para los estudiantes

1. Ejercicio 1. Crear una tabla llamada usuarios_por_carrera para consultar estudiantes según su carrera.
2. Ejercicio 2. Insertar mínimo 5 usuarios en usuarios_por_carrera.
3. Ejercicio 3. Consultar todos los usuarios de la carrera Sistemas.
4. Ejercicio 4. Crear una tabla prestamos_por_libro para consultar el historial de préstamos de un libro.
5. Ejercicio 5. Insertar mínimo 5 registros en prestamos_por_libro.
6. Ejercicio 6. Consultar todos los préstamos del libro L001.
7. Ejercicio 7. Actualizar el estado del préstamo P003 a Devuelto en la tabla prestamo_detalle.
8. Ejercicio 8. Eliminar un registro de libros_por_autor usando autor y titulo_libro.
9. Ejercicio 9. Explicar por qué Cassandra permite duplicar datos en varias tablas.

10. Ejercicio 10. Comparar brevemente Cassandra con PostgreSQL indicando dos diferencias importantes.

21. Evidencias que debe entregar el estudiante

- Captura de Cassandra iniciado correctamente.
- Captura de ingreso a cqlsh.
- Captura del keyspace creado.
- Captura de las tablas creadas.
- Captura de inserción de datos.
- Captura de mínimo 5 consultas SELECT.
- Captura de una actualización con UPDATE.
- Captura de una eliminación con DELETE.
- Desarrollo de los ejercicios propuestos.
- Conclusión personal de mínimo 8 líneas.

Nombre del archivo:

BDII_Actividad_Cassandra_NombreApellido

22. Rúbrica de evaluación sugerida

Criterio	Puntaje
Inicia Cassandra y accede correctamente a cqlsh	2 pts
Crea correctamente el keyspace	2 pts
Diseña y crea tablas orientadas a consultas	4 pts
Inserta correctamente los datos	3 pts
Realiza consultas SELECT usando claves adecuadas	3 pts
Actualiza y elimina registros correctamente	2 pts
Resuelve los ejercicios propuestos	2 pts
Presenta evidencias y conclusión clara	2 pts
Total	20 pts

23. Conclusión de la práctica

En esta práctica, el estudiante aplica los fundamentos de Cassandra DB mediante la creación de un keyspace, tablas orientadas a consultas, inserción de datos y operaciones básicas de consulta, actualización y eliminación. La actividad refuerza la idea central de Cassandra: diseñar primero según las consultas que necesita el sistema, evitando depender de relaciones y JOIN como en bases de datos relacionales.